Pipelined Processor

- Pipelining improves the throughput of a digital system.
- It is designed by subdividing the single-cycle processor into five pipeline stages.
- Five instructions can execute simultaneously, one in each stage.
- As each stage has only one-fifth of the entire logic, the clock frequency is almost five times faster.
- Hence, the latency of each instruction is ideally unchanged, but the throughput is ideally five-times better.

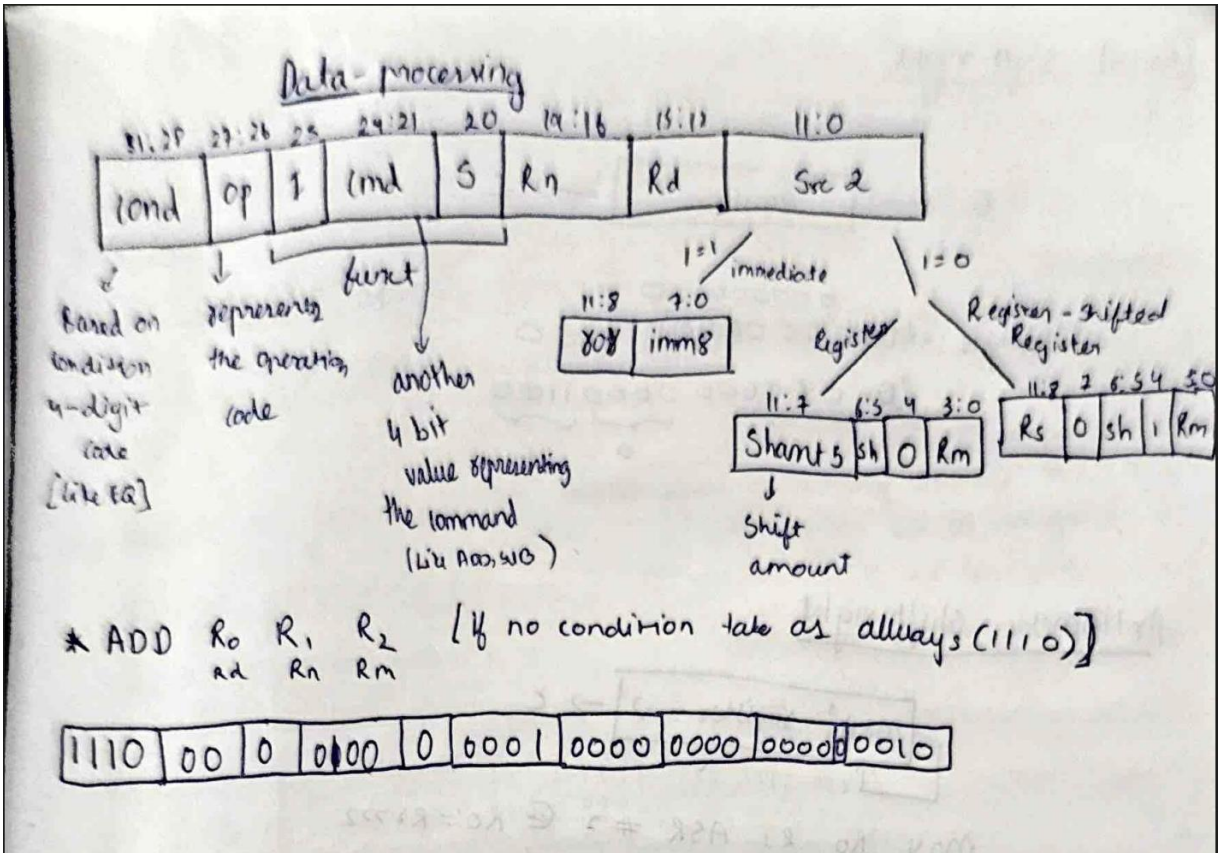
Pipelined Processor

Pipeline Stages

1) Fetch
   - Processor reads the instruction from instruction memory.
2) Decode
   - Processor reads the source operands from the register file and decodes the instruction to produce the control signals.
3) Execute
   - Processor performs a computation with the ALU.
4) Memory
   - Processor reads or writes the data memory.
5) Writeback
   - Processor writes the result to the register file, when applicable.

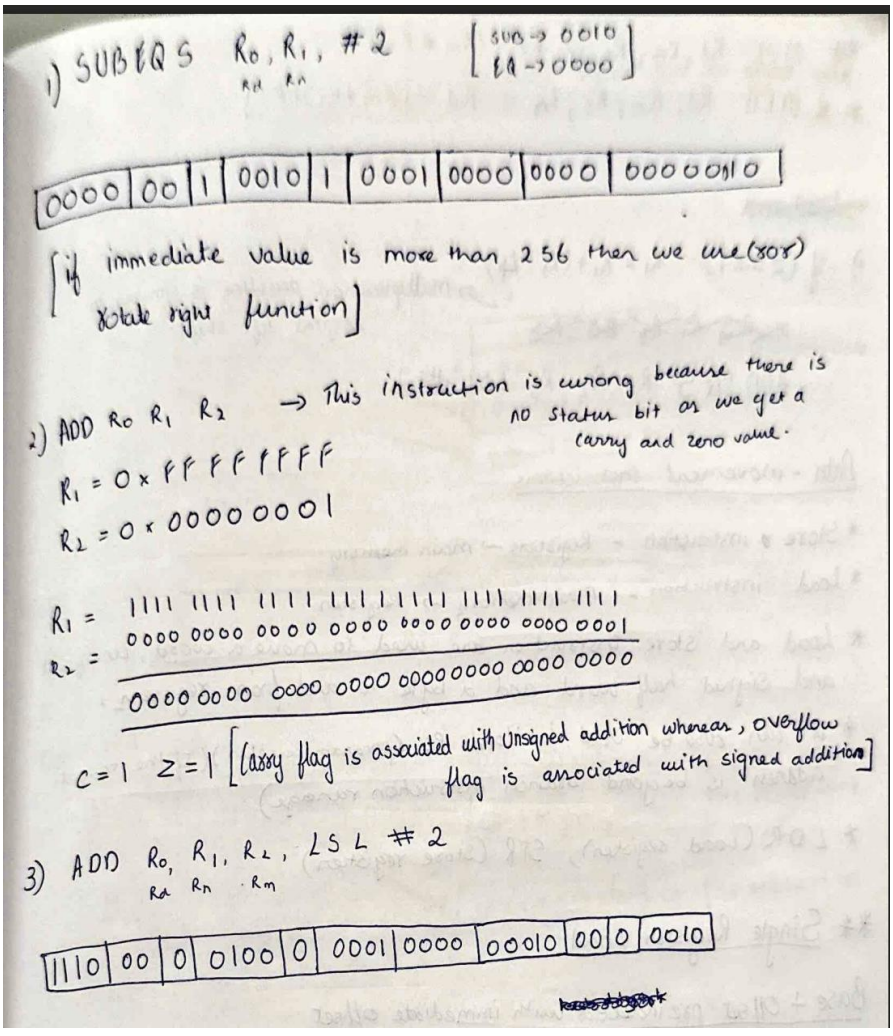
Pipelined Processor

- Each pipeline stage is represented with its five major component
  1) Instruction memory (IM),
  2) Register file (RF) read,
  3) ALU execution,
  4) Data memory (DM),
  5) Register file writeback

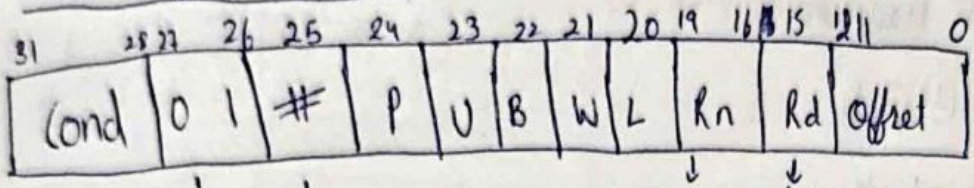


ADD Rd, Rn, Rm

ADD Rd, Rn #some immi value)



→ Data transfer instructions



op code

offset value

Base register

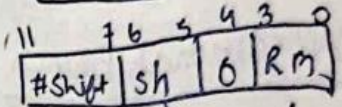
destination in load, source in store

[immediate value] → R1 [R0 #4]

[It is 1 when it is a register] → R2 [R0 R3 LSL #4]

= 0

= 1



Shift amount

type shift

offset register

00 - LSL
01 - LSR
10 - ASR
11 - RR

Flag bits

- L → single bit, represent if it is Load or store
  - L=0 → Store instruction
  - L=1 → Load instruction

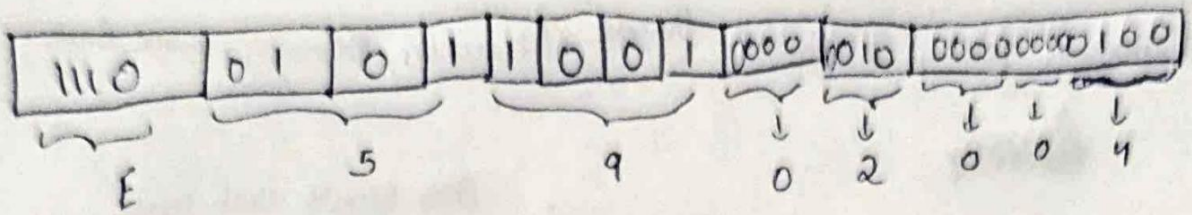
- W → W=0 → no write back [!]
  - W=1 → write back

- B → unsigned byte or word
  - B=0 → word
  - B=1 → byte

- U → increment or decrement (up or down)
  - U=0 → subtract offset from base
  - U=1 → add offset to the base

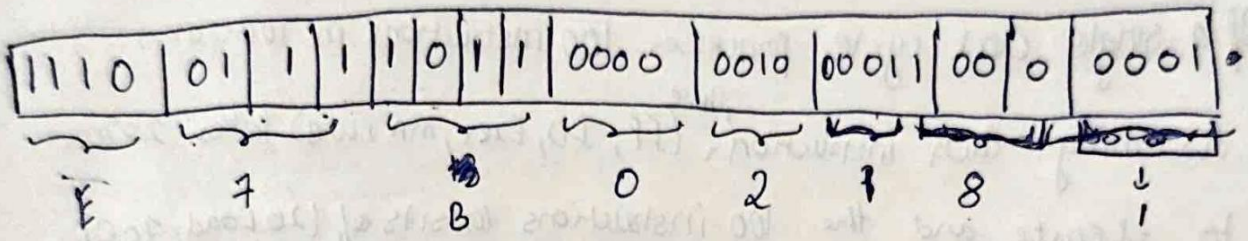
- P → postfix or prefix [post index or pre-index]
  - P=0 → Post
  - P=1 → Pre

Q) LDR R_2 [R_0 #4]

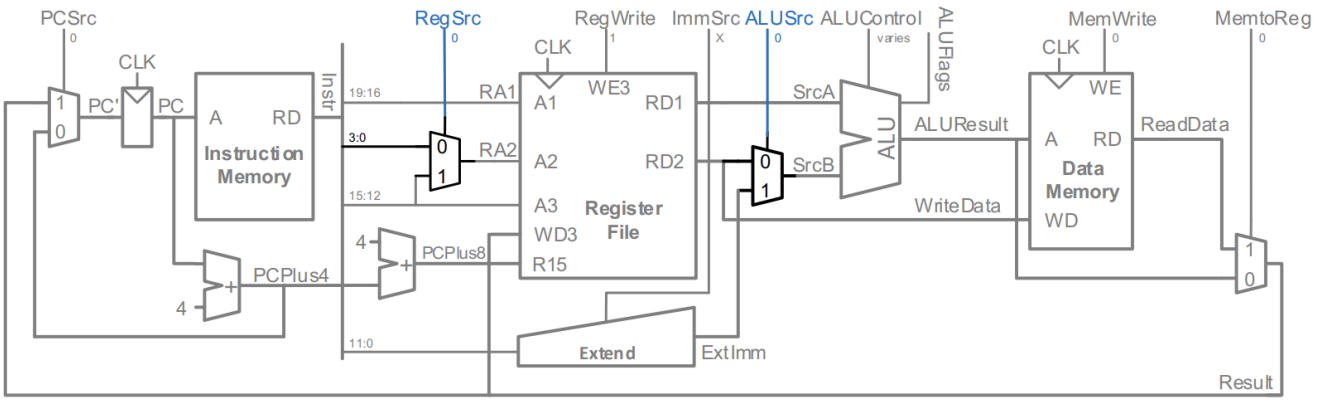


=> 0xE59020004 -> Hexadecimal format

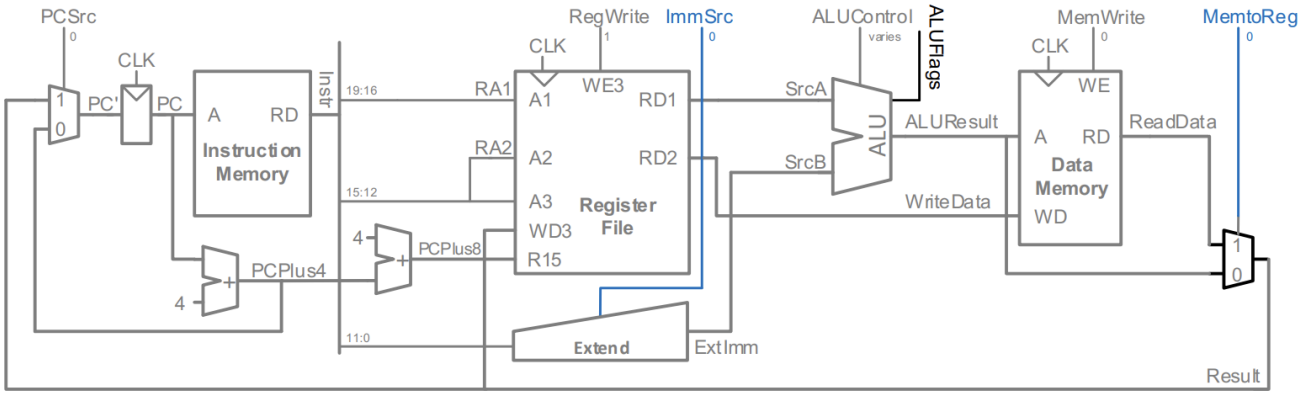
Q) LDR R_2 [R_0, R_1, LSL # 3]!



=> 0xE7B02181



ADD Rd, Rn, Rm



ADD Rd, Rn, imm8

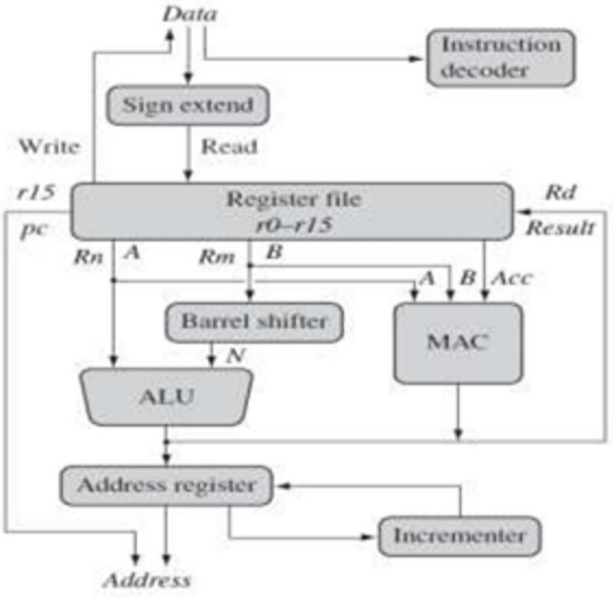


Figure 1.1 Data Flow Diagram of ARM